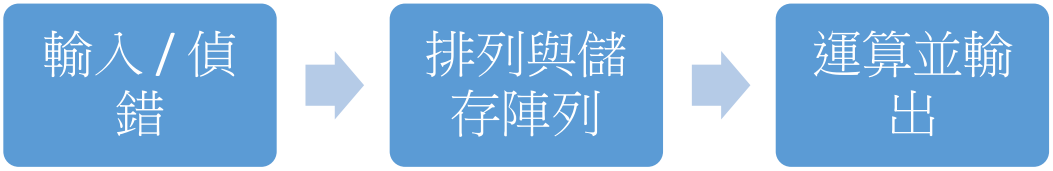
<PA3 Big number calculator>

< 程式運作原理 >

本程式的運作流程主要分為以下四個部分



為了執行以上的流程，分別定義了以下的陣列和函數，其中{ }內所定義的變數為該函數的區域變數。特別注意，由於各函數之間的相似度極高，連變數名稱都相同，這裡只會解釋 **add** 內部的變數及陣列名稱

1.**void add( )**{int aray[5][101],sum[3][102];char bray[3][101];}

用來處理加法的所有步驟，包括輸入，運算，輸出。

bray[1][ ]={a}:貯存輸入的第一筆資料(ex:1,2,3,0,0……)(char form)

bray[2][ ]={a}:貯存輸入得第二筆資料(ex:3,4,5,0,0……)(char form)

aray[1][ ]:將轉換後的第一筆資料存入(ex: 1,2,3,0,0……)(int form)

aray[2][ ]:將轉換後的第二筆資料存入(ex: 3,4,5,0,0……)(int form)

aray[3][ ]:將排序後的第一筆資料存入(ex: 0,0…..0,1,2,3)

aray[4][ ]:將轉換後的第二筆資料存入(ex: 0,0…..0,3,4,5)

sum[1][ ]:用來貯存相加的結果

sum[2][ ]:用貯存進位

2.**void sub( )**{ int aray[5][101],sub[3][102];char bray[3][101]; }

用來處理減法的所有步驟，包括輸入，運算，輸出。

Bray, aray:同 **add( )**

Sub[1][]:用來貯存相減的結果

Sub[2][]:用貯存借位

3 .**void mul( )**{ int aray[7][101];char bray[3][101]; }

用來處理乘法的所有步驟，包括輸入，運算，輸出。

Aray[1~4][ ],bray:同 **add( )**

Aray[5][ ]:用來貯存相乘的結果

Aray[6][ ]:用來貯存進位

4 . .**void mean(** char bray[3][101] **)**{ int aray[5][101],sum[3][102],mean[3][102]}

用來處理平均的所有步驟，包括輸入，運算，輸出。

Bray[1,2]:用來接收其他函數內 Bray[1,2]的值

Aray, sum:同 **add( )**

Mean[1][ ]:用來貯存平均的結果

Mean[2][ ]:用來貯存借位

<流程圖>

為了方便說明，以下用 5 digit 的運算來說明

Add():

(陣列經過每次操作的變化)

```
please enter a number
129
192
129
192

1 2 9 0 0 0 0 0 0 0   <----this is aray[1][]
1 9 2 0 0 0 0 0 0 0   <----this is aray[2][]
0 0 0 0 0 0 0 0 0 0   <----this is aray[3][]
0 0 0 0 0 0 0 0 0 0   <----this is aray[4][]

1 2 9 0 0 0 0 0 0 0   <----this is aray[1][]
1 9 2 0 0 0 0 0 0 0   <----this is aray[2][]
0 0 0 0 0 0 0 1 2 9   <----this is aray[3][]
0 0 0 0 0 0 0 1 9 2   <----this is aray[4][]


0 0 0 0 0 0 0 3 2 1   <----this is sum[1][]
0 0 0 0 0 0 0 1 1 0   <----this is sum[2][]

321
Process returned 0 (0x0)   execution time : 2.247 s
Press any key to continue.
```

NO.

DATE

<add>

假設輸入數字 129, 129

[129] input → bray[1][ ] = {1, 9, 2, a, a, a}

[129] input → bray[2][ ] = {1, 9, 2, a, a, a}

↓ convert to int form array a convert to 10

array[1][ ] = {1, 9, 2, 10, 10, 10}

array[2][ ] = {1, 9, 2, 10, 10, 10}

↓ 從右邊開始檢查，有 k (=2) 組相鄰，故向右移動 k+1 (=3) 格

array[3][k+i+1] = array[1][i]

array[3][k+i+1] = array[2][i]

↓

array[3][ ] = {10, 10, 10, 1, 9, 2}

array[4][ ] = {10, 10, 10, 1, 9, 2}

↓ 將 10 替換成 0

array[3][ ] = {0, 0, 0, 1, 9, 2}

array[4][ ] = {0, 0, 0, 1, 9, 2}

以上這些步驟為陣列的重新排序，將用於後面的 mul, sub, mean function.

由左而右

array[3][ ] = {0, 0, 0, 1, 9, 2}

array[4][ ] = {0, 0, 0, 1, 9, 2}

result: Sum[1][ ] = {0 0 0 3 8 4}

carry → Sum[2][ ] = {0 0 0 1 0 0}

被加數 加數 進位

Sum[1][i] = (array[3][i] + array[4][i] + Sum[2][i]) % 10

Sum[2][i-1] = (array[3][i] + array[4][i] + Sum[2][i]) / 10

(2 + 2 + 0) % 10 = 4

(2 + 2 + 0) / 10 = 0

(9 + 9 + 0) % 10 = 8

(9 + 9 + 0) / 10 = 1

(1 + 1 + 1) % 10 = 3

(1 + 1 + 1) / 10 = 0

0

array[3][ ] + array[4][ ] 得 Sum[1] 的這個方法稱為"陣列加法"，將用於 mul mean 函數

by the process, we can obtain the result stored in the Sum[1][ ].

為了印出首項不為 0 的結果，從左開始檢 Sum[1][ ] 有 l 組相鄰的 0，從 Sum[1][l+1] 開始列印到尾，即可得到答案。

Sum[1][ ] = {0, 0, 0, 3, 8, 4}, l = 2

print Sum[1][2+1] to Sum[1][5]

即可得 384

右邊的步驟稱為列印，同樣也會用於 mul, sub, mean, function

SEASON

(陣列重新排序)

```
for(int j=1;j<=2;j++)
{
   for(int i=0;i<101;i++)
    {
        aray[j][i]=bray[j][i];
        switch(aray[j][i])
        {
        case 48: aray[j][i+k]=0;break;
        case 49: aray[j][i+k]=1;break;
        case 50: aray[j][i+k]=2;break;
        case 51: aray[j][i+k]=3;break;
        case 52: aray[j][i+k]=4;break;
        case 53: aray[j][i+k]=5;break;
        case 54: aray[j][i+k]=6;break;
        case 55: aray[j][i+k]=7;break;
        case 56: aray[j][i+k]=8;break;
        case 57: aray[j][i+k]=9;break;
        default: aray[j][i+k]=10;
        }
    }
}
  for(int j=1;j<=2;j++) //the following three loop is aim to arrange the data from 12340000...to ...0001234
  {
     while(aray[j][100-k]==10&&aray[j][99-k]==10)k++;
   k++;
   for(int i=0;i<101-k;i++)
   {
      aray[j+2][k+i]=aray[j][i];
   }
   k=0;
  }

for(int j=1;j<=4;j++)
    {
        for(int i=0;i<101;i++)if(aray[j][i]==10)aray[j][i]=0;
    }
```

(列印)

```
while(sum[1][l]==0&&sum[1][l+1]==0)l++;//the following loop is going
      cout<<"Result: ";
      for(int i=l+1;i<102;i++)
      {
          cout<<sum[1][i];
      }
       cout<<endl;
```

Sub()

(陣列經過每次操作的變化)

```
please enter a number
192
129
192
129

1 9 2 10 10 10 10 10 10 10   <----this is aray[1][]
1 2 9 10 10 10 10 10 10 10   <----this is aray[2][]
0 0 0 0 0 0 0 0 0 0   <----this is aray[3][]
0 0 0 0 0 0 0 0 0 0   <----this is aray[4][]

1 9 2 0 0 0 0 0 0 0   <----this is aray[1][]
1 2 9 0 0 0 0 0 0 0   <----this is aray[2][]
0 0 0 0 0 0 0 1 9 2   <----this is aray[3][]
0 0 0 0 0 0 0 1 2 9   <----this is aray[4][]


0 0 0 0 0 0 0 0 6 3   <----this is sub[1][]
0 0 0 0 0 0 0 0 1 0   <----this is sub[2][]

63
Process returned 0 (0x0)   execution time : 2.861 s
Press any key to continue.
```

NO.

DATE

<Sub>

假定輸入 192, 129

192 → bray[1][ ]

129 → bray[2][ ]

⟹ 重新排列 → aray[3][ ] = {0, 0, 0, 1, 9, 2}

→ aray[4][ ] = {0, 0, 0, 1, 2, 9}

由右而左

被減數: aray[3][ ] = { 0, 0, 0, △, 9, 2 }

減數 aray[4][ ] = { 0, 0, 0, △, 2, 9 }

需要借位: Sub[1][i] = aray[3][i] - aray[4][i] - Sub[2][i] + 10

result → Sub[1][ ] = { 0, 0, 0, 0, 6, 3 } → 不需借位: Sub[1][i] = aray[3][i] - aray[4][i] - Sub[2][i]

Sub[2][ ] = { 0, 0, 0, 0, 1, 0 } → 需借位: Sub[2][i-1] + 1.

① (□-□-□) < 0 需借位

(□-□-□+10) = 3

② Sub[2][4] = (1)

③ (9-2-1) > 0 不需借位

(9-2-1) = 6

④ (△-△-△) > 0 不需借位

(△-△-△) = 0

by the process, we can obtain the result store in Sub[1][ ]

"列印" Sub[1][ ] 即可得 63 ☆

SEASON

Mul():

(陣列經過每次操作的變化)

```
Given 12345*12345
  0  0  0  0  0  1  2  3  4  5 *   5 + aray[5][] =   0  0  0  0  0  5 10 15 20 25<----New aray[5][]
aray[3][] after manuplate operation T
  0  0  0  0  1  2  3  4  5  0 *   4 + aray[5][] =   0  0  0  0  4 13 22 31 40 25<----New aray[5][]
aray[3][] after manuplate operation T
  0  0  0  1  2  3  4  5  0  0 *   3 + aray[5][] =   0  0  0  3 10 22 34 46 40 25<----New aray[5][]
aray[3][] after manuplate operation T
  0  0  1  2  3  4  5  0  0  0 *   2 + aray[5][] =   0  0  2  7 16 30 44 46 40 25<----New aray[5][]
aray[3][] after manuplate operation T
  0  1  2  3  4  5  0  0  0  0 *   1 + aray[5][] =   0  1  4 10 20 35 44 46 40 25<----New aray[5][]

  0  1  5  2  3  9  9  0  2  5<---This is result(mean[1][])
  0  0  1  2  3  4  5  4  2  0<----This is carry(mean[2][])


Process returned 0 (0x0)   execution time : 0.028 s
Press any key to continue.
```

(operation T)

```
for(int l=0;l< 2*k;l++)// operation T
      {
      aray[3][l]=aray[3][l+1];

      }
     aray[3][2*k-1]=0;
```

NO.

DATE

<mul>

若計算k位數的乘法，則需要2k-1個位元，為了方便解說，這裡將示範3位數的乘法。

假定輸入 129 × 128；　array[3][7]=[0, 0, 1, 2, 9]

array[4][4]=[0, 0, 1, 2, 8]

(main concept:)

129 × 128可表示成

$$\begin{bmatrix} 0 & 0 & 1 & 2 & 9 \\ 0 & 1 & 2 & 9 & 0 \\ 1 & 2 & 9 & 0 & 0 \end{bmatrix} \begin{bmatrix} 8 & 2 & 1 \\ 8 & 2 & 1 \\ 8 & 2 & 1 \\ 8 & 2 & 1 \\ 8 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 8 & 16 & 72 \\ 0 & 2 & 4 & 18 & 0 \\ 1 & 2 & 9 & 0 & 0 \end{bmatrix}$$ ⇒ 三列相加

為了實現這個矩陣，我們設計一個操作下能將每一列的元素向左1格，並在最後一格補0

⇓

[1, 4, 21, 34, 72]，與[0, 0, 0, 0, 0]"作陣列加法"即可得[1, 6, 5, 1, 2]

(詳細操作原理請對照 add function)

[1, 4, 21, 34, 72] → 類比為 add function array[3][ ]

[0, 0, 0, 0, 0] → 類比為 add function array[4][ ]

"列印"出來 ← [1, 6, 5, 1, 2] → 類比為 add function Sum[1][ ] (result)

[0, 2, 4, 7, 9] → 類比為 add function Sum[2][ ] (carry)

(程式落實)

array[3][ ]

(0, 0, 1, 2, 9) × array[4][4] (=8) = array[5][ ] = (0, 0, 8, 16, 72) array[5][ ]

array[3]轉T　　代入

(0, 1, 2, 9, 0) × array[4][3] (=2) + array[5] = (0, 2, 4, 18, 0) + (0, 0, 8, 16, 72)

= (0, 2, 12, 34, 72) new array[5][ ]

array[3][ ]　　代入

(1, 2, 9, 0, 0) × array[4][2] (=1) + array[5] = (1, 2, 9, 0, 0) + (0, 2, 12, 34, 72)

= (1, 4, 21, 34, 72) new array[5][ ]

按照 main concept，可得出結果 16512

SEASON

NO.

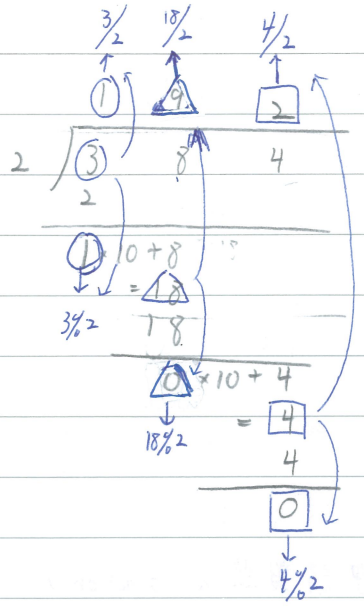
DATE

<mean>

假定輸入 192, 192

利用 add function，我們可以得到一陣列 Sum[1][] = [0, 0, 0, 3, 8, 4]

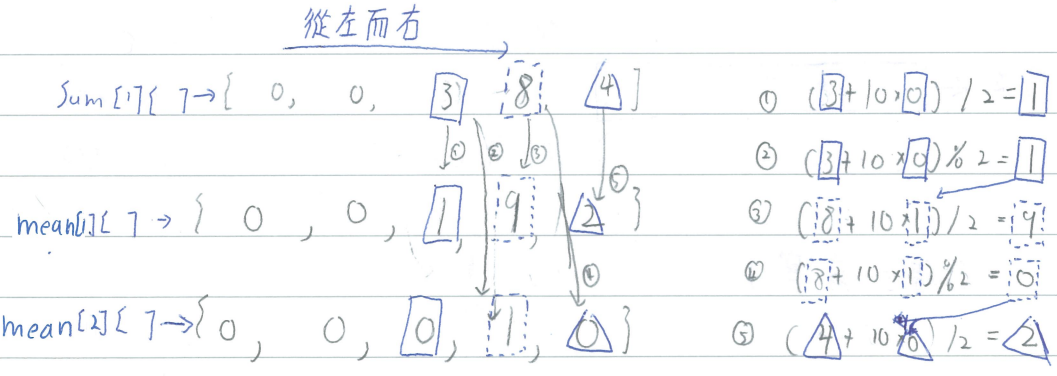
= 192 + 192



(main concept)

利用長除法的原理，我們可以做到取平均的運算。

(程式的落實



mean[1][i] = (Sum[1][i] + mean[2][i]) /2

mean[2][i+1] = (Sum[1][i] + mean[2][i]) %2

⇓

"列印"陣列 mean[1][] 即可得結果 192

SEASON

<困難的地方>

最先遇到的困難是如何輸入數字，當時在設計的時候還沒有學到 string 和 vector，只能使用字元陣列來處理，因此浪費大量的記憶體去貯存沒有運算的位數。接著就是在設計程式時，我對於函數如何傳遞陣列也不熟，再加上我是每個函數分開設計的，所以最後全部組裝起來的時候讓整個程式變得十分龐大。現在回頭來看應該還有更精簡更有效率的作法才對。還有一開始設計程式時，我是讓 bray 的每個初始值為零，然而這就造成了我的程式分辨不出 100 和 1000 有什麼差別，為了改善這個問題，最後才想到將 bray 的初始值設為 10 再來檢查有幾組 10 相同。最後我透過編寫這個程式得到了一個結論，一個複雜的程式很難兼具運算速度和記憶容量，在寫 mul function 時我特別感受到了這個問題。